Skip List Challenge

Giacomo Rigo Student ID: 308362

1. Structure and expected costs

The list stores distinct signed 64-bit keys. Each node has a vector of forward pointers. Heights use promotion probability $p = 1/2$, capped at 32. The head has 32 pointers; its dummy key is never compared, so key 0 and both 64-bit boundary keys are valid.

Search moves right and down through levels that become about twice as sparse. This gives expected $O(\log(n+2))$ time in the supported range. Insert uses the same search and changes fewer than two levels on average, so it has the same expected bound. Ordered union makes two descents to the ends of A, including its order check, then joins the levels: expected $O(\log(n+m+2))$. Split finds the predecessors of the pivot and cuts each level: expected $O(\log(n+2))$. Header creation, generator setup, and resetting the 32 head links are included; none scans all data nodes.

Union requires distinct inputs and all keys of A below those of B when both are nonempty. Split returns keys $\le x$ and $> x$. Both transfer nodes without changing addresses, keys, or heights. Consumed inputs stay empty and reusable. Outputs own their nodes separately and free them once. The mean height is below 2, so expected storage is $O(n)$. For $n \le 2^{20}$, the chance that any uncapped height exceeds 32 is at most $2^{20}/2^{32} = 1/4096$. The fixed cap does not support a logarithmic claim for arbitrary sizes.

2. Checks and experiment setup

Tests compare with `std::set` and check order, levels, links, cycles, node identity, and independent ownership. They cover empty/singleton lists, duplicates, boundary keys, unequal unions, split positions, and reuse. There are 1000 mixed operations per seed (11, 42, 2026), including repeated split/union. The final Linux rebuild (GCC 14.2) passed the tests and ASan/UBSan with leak detection; logs are in `results`.

The timing data below are from Apple M2, macOS 26.6.2, Apple Clang 21.0.0, SDK 26.5, and Google Benchmark 1.10.0. Builds use C++17, `-O2 -DNDEBUG`, warning flags, and `-pthread`. Full commands are in the README. Six sizes $N = 2^{10}, 2^{12}, \ldots, 2^{20}$ use three height seeds and three repeats, on one thread. B uses seed +1000003. Separate queries use seed $20261023 + N$, uniform sampling with repeats, and 50% hits. Union has two lists of $N/2$ keys; split uses a middle pivot. Insert uses absent keys and restores the size after 64 inserts.

Fixed batches contain 256 searches or 64 inserts; other batches contain one call. Iterations are bounded (256, 1000, or 3 for rebuild). Setup, checks, and restoration are outside timing; operation allocations and frees are inside. Counters are disabled. Barriers keep the measured work from being removed. Times and fit coefficients are divided by batch size.

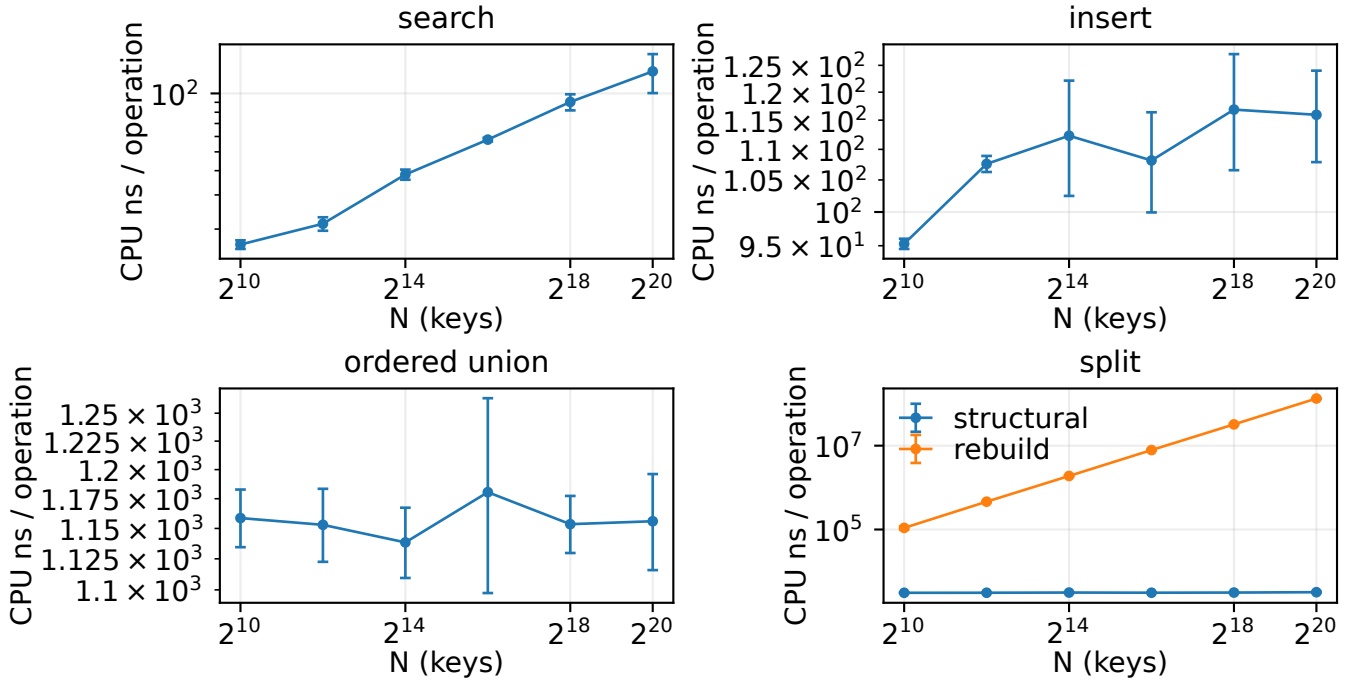


Figure 1. Mean CPU time per operation; bars show standard deviation over nine samples (three seeds, three repeats). All six sizes are shown. Both axes are logarithmic.

3. Timing fits and the rebuilding comparison

Google Benchmark selects each model automatically using `Complexity()` and `SetComplexityN(N)`. Table 1 reports $t(N) \approx Cf(N)$ for one operation; RMS is the normalized fit error in percent. Coefficients are rounded.

Operation	Seed	$f(N)$	C (ns)	RMS (%)
Search	11	N	1.6734×10^{-4}	55.43
Search	42	$\log_2 N$	4.1793	43.60
Search	2026	$\log_2 N$	3.9822	40.71
Insert	11	1	109.76	10.82
Insert	42	1	111.33	9.57
Insert	2026	1	106.97	7.23
Ordered union	11	1	1144.3	2.66
Ordered union	42	1	1193.6	4.40
Ordered union	2026	1	1132.2	1.71
Split	11	1	3154.3	3.14
Split	42	1	3119.3	1.54
Split	2026	1	3103.7	1.23
Rebuild split	11	N	126	2.55
Rebuild split	42	N	125.9	1.70
Rebuild split	2026	$N \log_2 N$	6.4797	3.83

Table 1. Automatic fits; $\log_2$ is the logarithm used by the library.

The baseline traverses the input, creates fresh nodes through ordinary insertions, and frees the consumed nodes. Its expected cost is $O(n \log(n+2))$; the structural split only changes links along the search path. Both use the same generated inputs and pivot. At $N = 2^{20}$, their mean times are 3.194 microseconds (structural) and 133.355 milliseconds (rebuild). This difference describes these workloads, not a general speedup guarantee.

Separate instrumented runs count key comparisons, steps to data nodes, and writes to `next`, including initialization and head links. Table 2 adds these events and normalizes each batch. Allocation/free counts include only data nodes.

Operation	Work at $N = 1024$	Work at $N = 1048576$
Search	30.99	56.52
Insert	26.21	47.54
Ordered union	138.67	159.67
Split	294.33	330.67
Rebuild split	24,866.67	47,896,896

Table 2. Mean logical events per operation over the three height seeds.

Structural search, union, and split allocate and free zero data nodes. Insert allocates one per call. Rebuild allocates and frees N nodes. Structural operations still create a constant number of headers. This explains why rebuild work grows strongly while structural split work changes little.

4. Interpretation and limits

A constant fit for insert, union, or split does not prove $O(1)$ time. Fixed head and generator costs can hide a small logarithmic increase. Search time rises from 16.65 to 129.94 ns, but its work count rises only from 30.99 to 56.52. Its 41–55% fit error shows that the timing trend is noisy; the linear fit for seed 11 does not prove linear search. Cache effects and run variability may contribute, but these data do not isolate a cause. The rebuild fits also vary between N and $N \log_2 N$; neither replaces the algorithm analysis.

The study uses only three seeds, a fixed height cap, and insertions above the maximum. It does not cover all insertion positions or input distributions. The Mac log warns that thread affinity could not be set, which limits timing control. The reported CPU-frequency metadata are unreliable and are not used. The counts support the structural design, while the timing fits need these limits to be understood.

Sources: course assignment and lectures; Google Benchmark official guide.

URL: `github.com/google/benchmark`. AI assistance is declared in the README.